

```
In [38]: import pandas as pd
        from sklearn.datasets import fetch_california_housing
        from sklearn.decomposition import PCA
        from sklearn.ensemble import RandomForestRegressor
        from sklearn.inspection import permutation_importance
        from sklearn.linear_model import LinearRegression
        from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
        from sklearn.model_selection import train_test_split
        from sklearn.pipeline import Pipeline
        from sklearn.preprocessing import StandardScaler
```

```
In [39]: data_housing = fetch_california_housing()
```

```
In [40]: data_housing
```

```

Out[40]: {'data': array([[ 8.3252, 41., 6.98412698, ..., 2.55555556,
    37.88, -122.23],
    [ 8.3014, 21., 6.23813708, ..., 2.10984183,
    37.86, -122.22],
    [ 7.2574, 52., 8.28813559, ..., 2.80225989,
    37.85, -122.24],
    ...,
    [ 1.7, 17., 5.20554273, ..., 2.3256351,
    39.43, -121.22],
    [ 1.8672, 18., 5.32951289, ..., 2.12320917,
    39.43, -121.32],
    [ 2.3886, 16., 5.25471698, ..., 2.61698113,
    39.37, -121.24]], shape=(20640, 8)),
'target': array([4.526, 3.585, 3.521, ..., 0.923, 0.847, 0.894], shape=(20640,)),
'frame': None,
'target_names': ['MedHouseVal'],
'feature_names': ['MedInc',
'HouseAge',
'AveRooms',
'AveBedrms',
'Population',
'AveOccup',
'Latitude',
'Longitude'],
'DESCR': '.. _california_housing_dataset:\n\nCalifornia Housing dataset\n-----\n\n**Data Set C
haracteristics:**\n\nNumber of Instances: 20640\n\nNumber of Attributes: 8 numeric, predictive attributes and the
target\n\nAttribute Information:\n    - MedInc        median income in block group\n    - HouseAge        median hous
e age in block group\n    - AveRooms        average number of rooms per household\n    - AveBedrms        average number
of bedrooms per household\n    - Population    block group population\n    - AveOccup        average number of househo
ld members\n    - Latitude    block group latitude\n    - Longitude    block group longitude\n\nMissing Attribut
e Values: None\n\nThis dataset was obtained from the StatLib:\nhttps://lib.stat.cmu.edu/datasets/houses.zip\n\nThe t
arget variable is the median house value for California districts,\nexpressed in hundreds of thousands of dollars
($100,000).\n\nThis dataset was derived from the 1990 U.S. census, using one row per census\nblock group. A block gr
oup is the smallest geographical unit for which the U.S.\nCensus Bureau publishes sample data (a block group typicall
y has a population\nof 600 to 3,000 people).\n\nA household is a group of people residing within a home. Since the
average\nnumber of rooms and bedrooms in this dataset are provided per household, these\ncolumns may take surprising
ly large values for block groups with few households\nand many empty houses, such as vacation resorts.\n\nIt can be
downloaded/loaded using the\nfunc:`sklearn.datasets.fetch_california_housing` function.\n\n.. rubric:: References\n
\n- Pace, R. Kelley and Ronald Barry, Sparse Spatial Autoregressions,\nStatistics and Probability Letters, 33:291-
297, 1997.\n'}

```

```
In [41]: X = data_housing.data
```

```
In [42]: y = data_housing.target
```

```
In [43]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20, random_state = 42)
```

```
In [44]: linear_regression = Pipeline(steps = [("scaler", StandardScaler()), ("regressor", LinearRegression()), ]) # base model
```

```
In [45]: random_forest = RandomForestRegressor(n_estimators = 300, min_samples_leaf = 2, random_state = 42, n_jobs = -1)
```

```
In [46]: pca_random_forest = Pipeline(steps = [("scaler", StandardScaler()), ("pca", PCA(n_components = 0.95, svd_solver = "full"))])
```

```
In [47]: models = {"Linear Regression (baseline)": linear_regression, "Random Forest": random_forest, "PCA + Random Forest": pca_random_forest}
```

```
In [48]: results = []
for model_name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    results.append(
        {
            "Model": model_name,
            "MAE": mean_absolute_error(y_test, predictions),
            "RMSE": mean_squared_error(y_test, predictions) ** 0.5,
            "R2": r2_score(y_test, predictions),
        }
    )
```

```
In [49]: comparison = pd.DataFrame(results).sort_values("RMSE").reset_index(drop = True)
print("Model comparison:")
print(comparison.to_string(index = False, float_format = "{:.4f}".format))
```

Model comparison:

	Model	MAE	RMSE	R2
	Random Forest	0.3263	0.5039	0.8062
	PCA + Random Forest	0.4746	0.6796	0.6476
	Linear Regression (baseline)	0.5332	0.7456	0.5758

As per these metrics, Random Forest (without PCA) has the lowest errors and should be chosen as the right model to use; as the Random Forest with PCA has a slightly higher amount of error, which means that most of the features play a part in this model and

shouldn't be reduced.

```
In [50]: fitted_pca = pca_random_forest.named_steps["pca"]
print(f"PCA components retained: {fitted_pca.n_components}")
print(f"Explained variance retained: {fitted_pca.explained_variance_ratio_.sum():.2%}")
```

PCA components retained: 6
Explained variance retained: 98.37%

```
In [51]: importance = permutation_importance(pca_random_forest, X_test, y_test, scoring = "neg_root_mean_squared_error", n_re
```

```
In [53]: feature_names = data.feature_names
```

```
In [56]: feature_importance = pd.DataFrame(
    {
        "Feature": feature_names,
        "Importance (RMSE increase)": importance.importances_mean,
        "Std. deviation": importance.importances_std,
    }
).sort_values("Importance (RMSE increase)", ascending = False)
```

```
In [ ]: feature_importance
```

```
Out[ ]:
```

	Feature	Importance (RMSE increase)	Std. deviation
--	---------	----------------------------	----------------

0	MedInc	0.609429	0.011040
6	Latitude	0.130368	0.004989
1	HouseAge	0.112622	0.006543
7	Longitude	0.111024	0.004751
5	AveOccup	0.107050	0.003905
2	AveRooms	0.045045	0.003932
4	Population	0.029703	0.002688
3	AveBedrms	0.009999	0.003702

MedInc has the most influential predictor, followed by Latitude, HouseAge, Longitude and AverageOccupation